MODULE 3 · MACHINE LEARNING · DAY 36

Honest Scores

Train/test split, cross-validation, pipelines

Today is not about building a better model.
It is about finding out whether you have one.

5	19	0
sections	worked pictures	new models

Srinivasa Sai Chava · Boston University

WHERE WE ARE

Two things yesterday left unfinished

Practice B4

You split 52 patients six different ways. The accuracy came out anywhere from 0.5625 to 0.8125 — on the same model and the same data.

The honest conclusion was: one split of a small dataset is not a measurement.

Every single day so far

You have written train_test_split at the top of every notebook since Day 32, because I told you to.

Today you find out what it is actually protecting you from — and where it is not enough.

Today's five sections

1. Marking your own homework
2. One split is a lottery
3. Give every row a turn
4. The leak you cannot see
5. One object, no leaks

No new model today.

Everything here applies just as much to linear regression from Day 33 as to the trees and forests coming next week. These are the habits, and you will use them for the rest of the course.

A score is a claim about rows you have not seen yet.

Every number you have reported so far was supposed to answer one question: if I use this model tomorrow, on somebody new, how often will it be right? Today is about measuring that honestly.

SECTION 1 OF 5

Marking your own homework

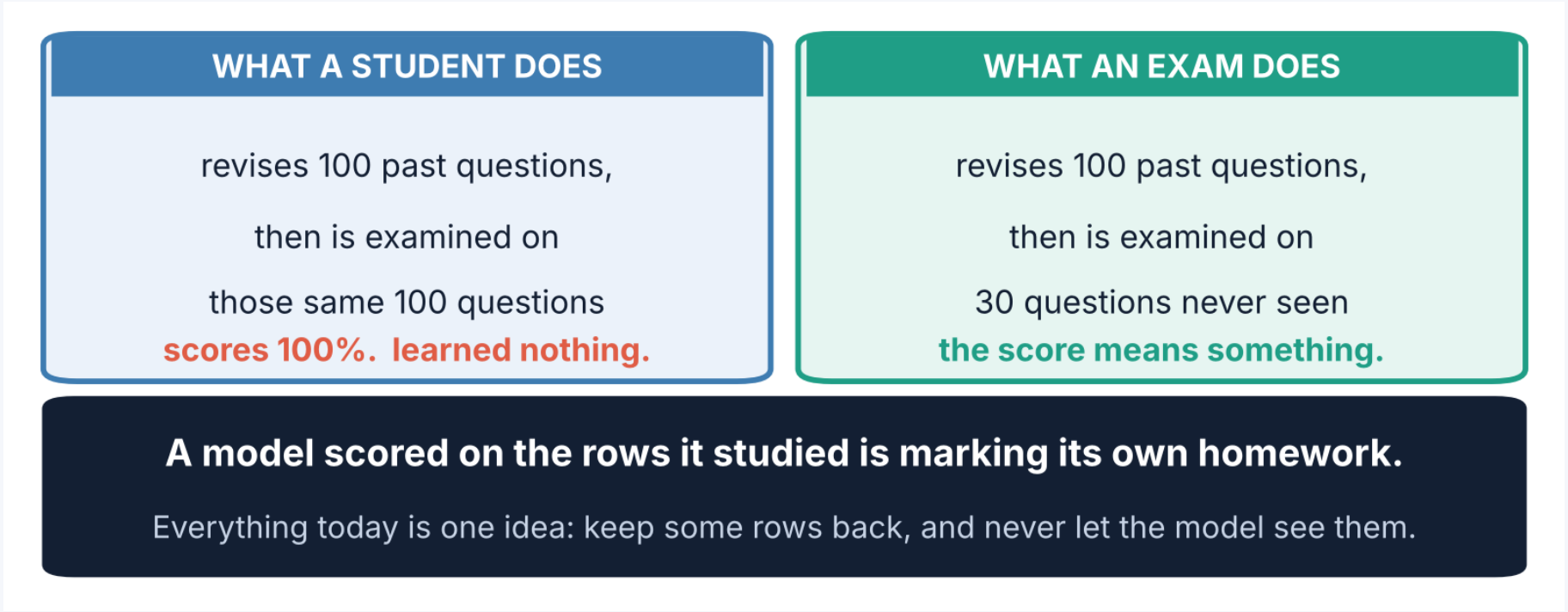
Why a score on the rows the model studied is not a score at all

- A tree that was 100% right
- and worse than guessing
- overfitting, measured

Srinivasa Sai Chava · Boston University

THE PICTURE

The whole idea, before any code



This is not an analogy that breaks down later. It is exactly what happens: a flexible model can store the answers instead of learning the pattern.

120 STUDENTS

The data we will use all morning

```
rng = np.random.default_rng(3601)
n = 120

hours      = rng.uniform(0, 20, n).round(1)
attendance = rng.uniform(40, 100, n).round(0)
prev_score = rng.normal(62, 14, n).clip(25, 98).round(0)

#  passed = 1 if the student passed the exam
```

120

students

3

columns about them

75%

of them passed

Small on purpose

120 rows is a realistic size for a class project, a pilot study, or the first month of any new product.

It is also small enough that every problem in today's lecture shows up clearly. On 200,000 rows most of them hide — which is exactly why people never learn them.

75% of these students passed, so 'everybody passes' is already right three times in four. Hold on to that number.

TWO MODELS, SAME ROWS

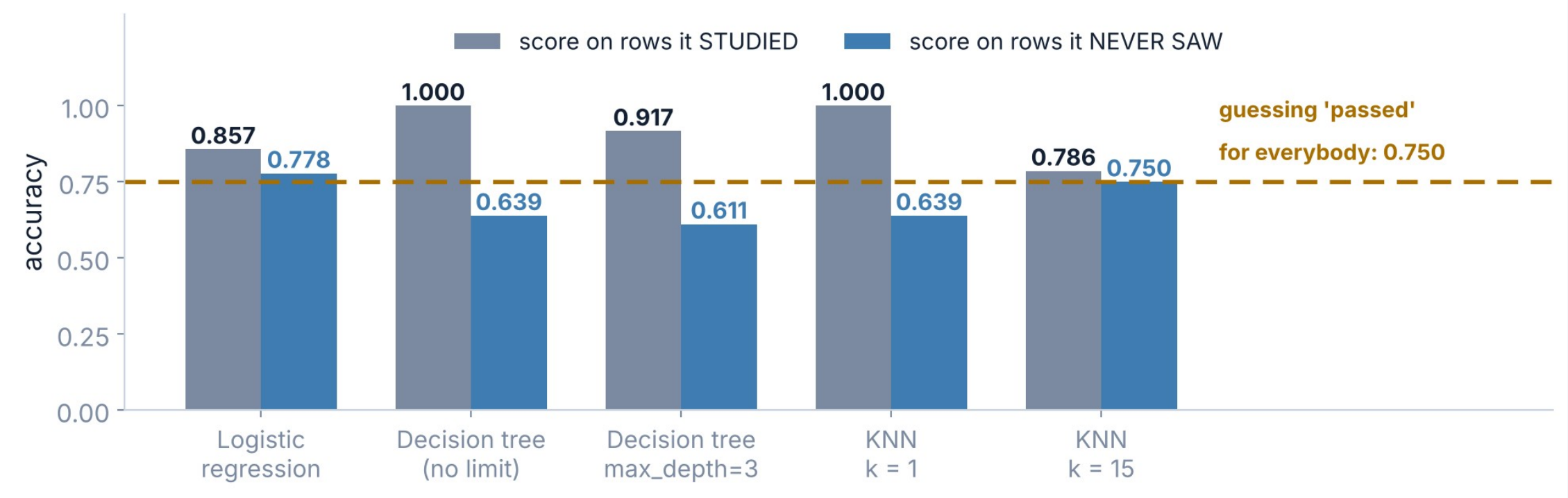
Memorising and learning look different



k = 1 asks 'who is my single closest classmate?' — so every student is trivially right about themselves. k = 15 asks fifteen, and has to accept being wrong about some of them.

THE GAP IS THE STORY

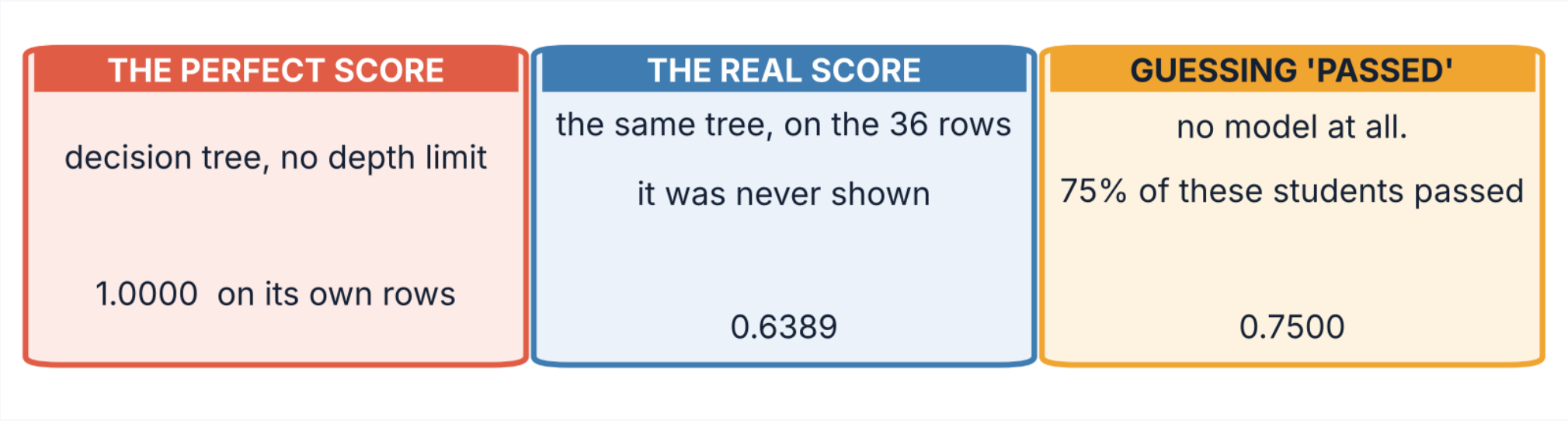
Five models, two scores each



Read the grey bars alone and the decision tree is the best model in the room. Read the blue bars and it is the worst — beaten by having no model at all.

100% THAT MEANS NOTHING

The same tree, three ways of asking

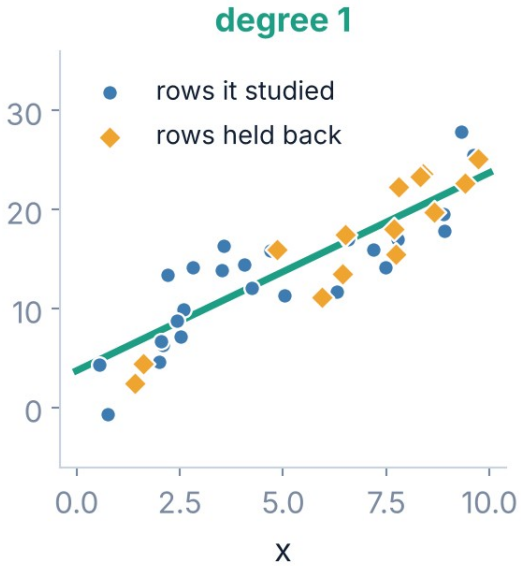


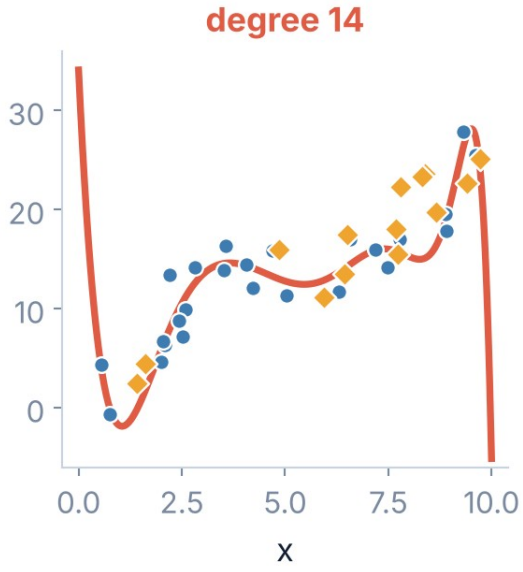
The tree grew one branch for every student it was shown. Asked about those same students it looks them up and is never wrong. Asked about anybody else it has nothing to go on.

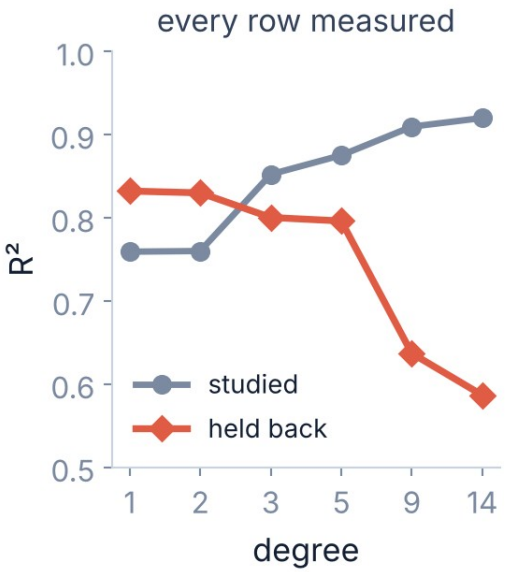
A model that scores 1.0000 on its own rows has told you nothing except that it was flexible enough to memorise them.

DEGREE BY DEGREE

The same thing in regression







studied R^2 = 0.7590

held back R^2 = 0.8319

studied R^2 = 0.9197

held back R^2 = 0.5865

one line goes up every time.

the other goes down every time.

Add flexibility and the score on the studied rows rises every single time. That is not evidence of anything. The held-back score fell every single time.

THE DEFINITION

Overfitting, in plain words

A model overfits when it has learned things that are true of the rows it studied and not true of anything else.

What makes it more likely	How you detect it	What you do about it
a flexible model (deep tree, k = 1, degree 14)	score the rows it studied	a simpler model
few rows	score rows it never saw	more rows
many columns	look at the gap	fewer columns
noisy data	there is no other way	regularisation — a day of its own, later

Notice the middle column. Detecting overfitting costs you nothing but discipline, and no amount of cleverness replaces it.

AND WHAT IT DOES

The line you have been writing since Day 32

```
from sklearn.model_selection import train_test_split

X_train, X_hidden, y_train, y_hidden = train_test_split(
        X, y, test_size=0.3, random_state=0, stratify=y)
```

argument	what it does	what happens without it
test_size=0.3	holds back 30% of the rows	defaults to 0.25
random_state=0	makes the split repeatable	a different split every run, so your results change
stratify=y	keeps the class mix the same on both sides	the mix drifts, badly on small or imbalanced data
shuffle=True	mixes the rows first (the default)	a sorted file splits into two different worlds

Three of these four arguments exist to stop the split itself from lying to you. That is the subject of the next two sections.

SECTION 2 OF 5

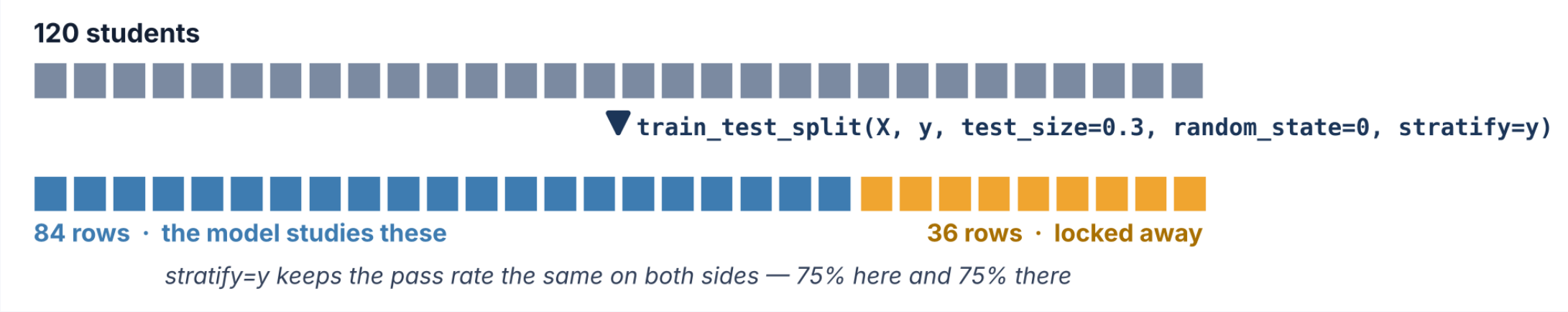
One split is a lottery

The split protects you from overfitting, but it brings a problem of its own

- 17 points from a seed
- which number do you report?
- why more rows help

120 ROWS, CUT IN TWO

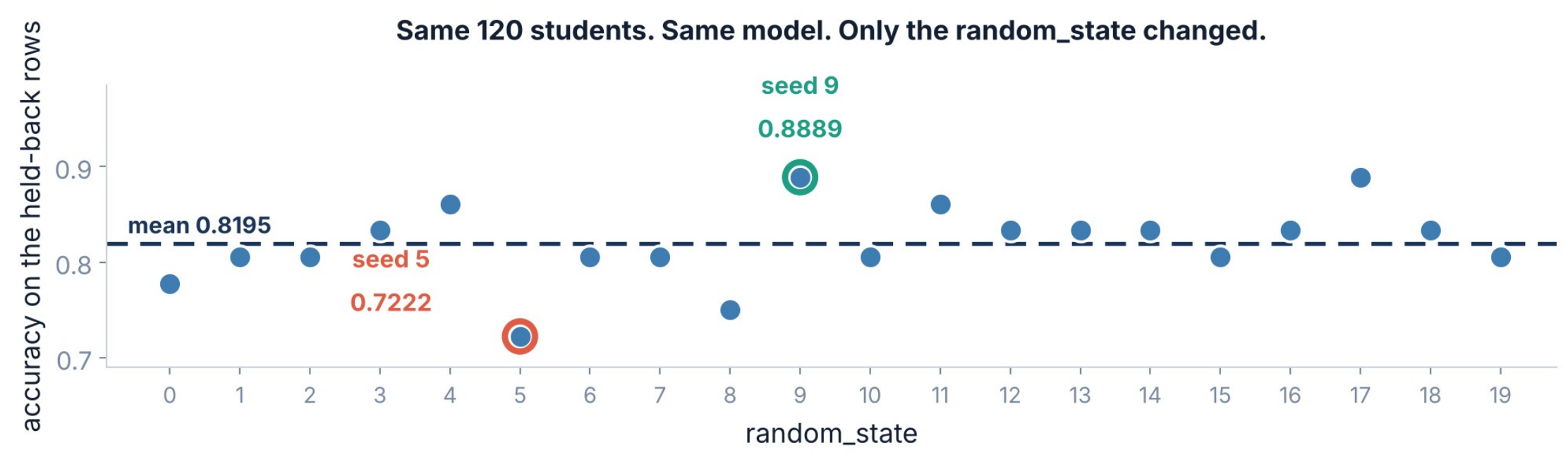
What the split actually does



The 36 held-back rows are the entire basis for everything you will claim about this model. Thirty-six rows.

THE LOTTERY

Run it twenty times. Change nothing but the seed.



Nothing about the model changed between these twenty dots. Not one coefficient, not one row of data. Only which 36 students happened to land in the held-back pile.

AN UNCOMFORTABLE QUESTION

So which number do you report?

Report 0.8889

Pick the best seed. This is how a great many published results happen, usually without anyone intending it — you try a few splits, one looks good, you stop trying.

Report 0.7222

Pick the worst seed. Honest in spirit, but you are now throwing away a model that is probably better than that.

Report 0.8194

The average of all twenty. Closer to the truth — and the idea behind everything in the next section.

The real problem is not which one to pick.

It is that a single split gives you one number and no idea how much to trust it. 0.8056 might mean "about 0.80" or it might mean "somewhere between 0.72 and 0.89". One split cannot tell you which.

You are not choosing between honest and dishonest here. You are choosing between a measurement and a guess.

THE ARITHMETIC

Why 36 rows swing so hard

One student is worth 2.8 points

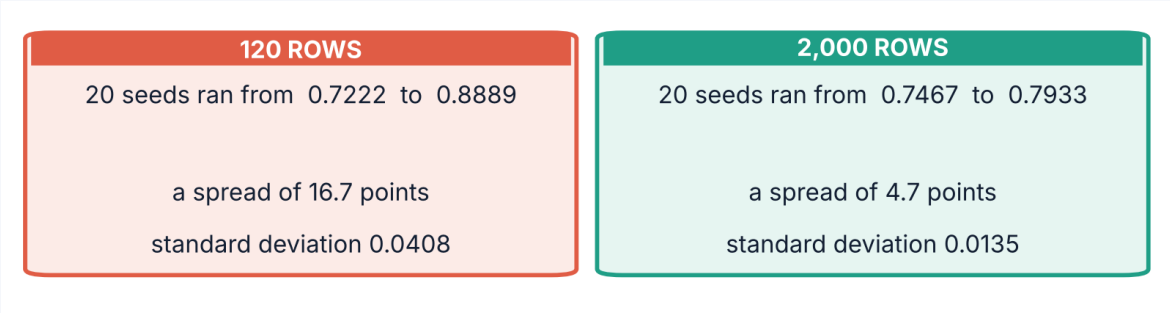
With 36 held-back rows, every single student the model gets right or wrong moves the reported accuracy by 1/36 = 0.0278.

Six students' difference is the whole 16.7-point spread. Six students, out of a hundred and twenty.

And the training half moves too

It is not only the exam that changes. Each seed also hands the model a different 84 students to learn from, so you are measuring a slightly different model each time.

Both halves are rolling dice at once.



More rows is the only fix that needs no new technique — and the one you usually cannot have. Everything else today works with the rows you already own.

SECTION 3 OF 5

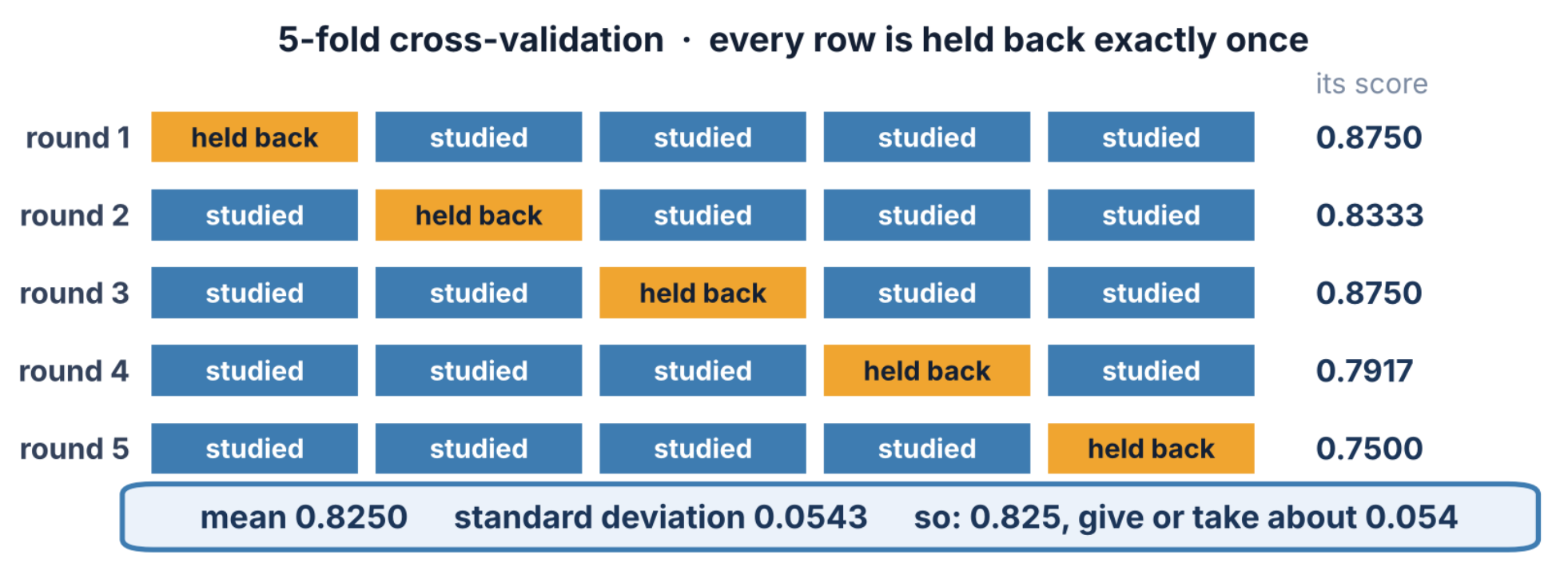
Give every row a turn

If one exam is unreliable, sit five of them and average the marks

- k-fold, drawn out
- how many folds
- the sorted-file disaster

FIVE ROUNDS

Cross-validation, drawn out in full



Every row is held back exactly once and studied four times. Nothing is wasted, and no row is ever scored by a model that saw it.

CROSS_VAL_SCORE

Three lines of code

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv     = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(LogisticRegression(max_iter=2000), X, y, cv=cv)

scores   ->  [0.8750  0.8333  0.8750  0.7917  0.7500]
```

Note what you pass in

All of X and all of y — not X_train. Cross-validation does its own splitting, five times over, and you must not have split the data yourself first.

It also fits five separate models and throws all five away. What you keep is the five numbers.

How to report it

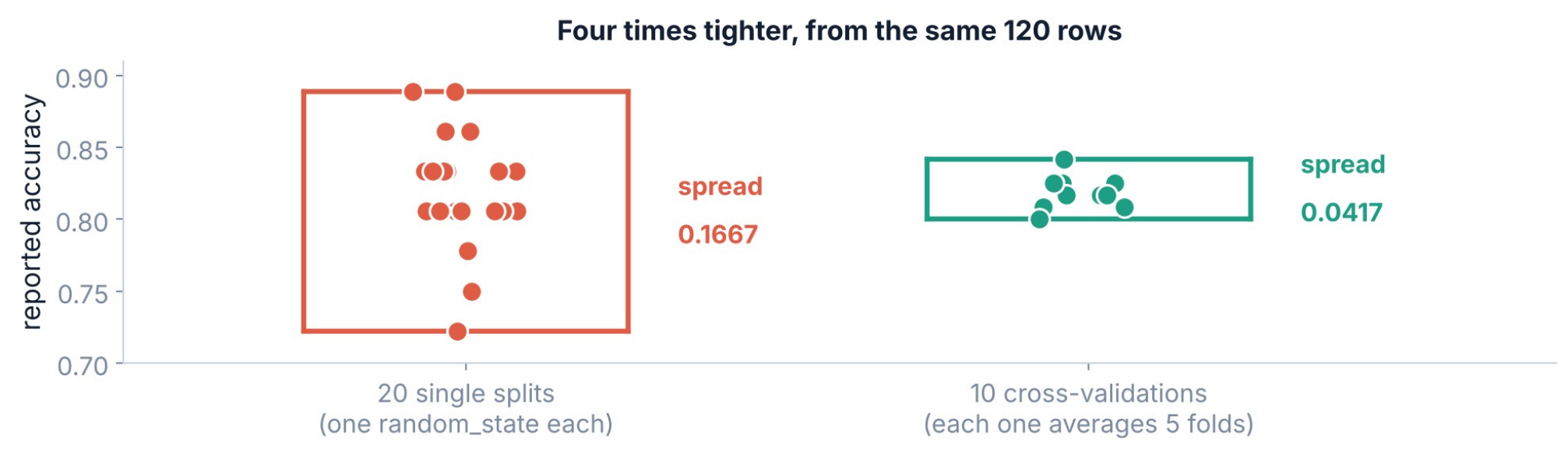
mean 0.8250
standard deviation 0.0543

Say it out loud as: "about 0.83, give or take about 0.05." Never quote the mean on its own — the spread is half the result.

Five numbers, not one. If they disagree wildly with each other, that is the result telling you something.

CV AGAINST SINGLE SPLITS

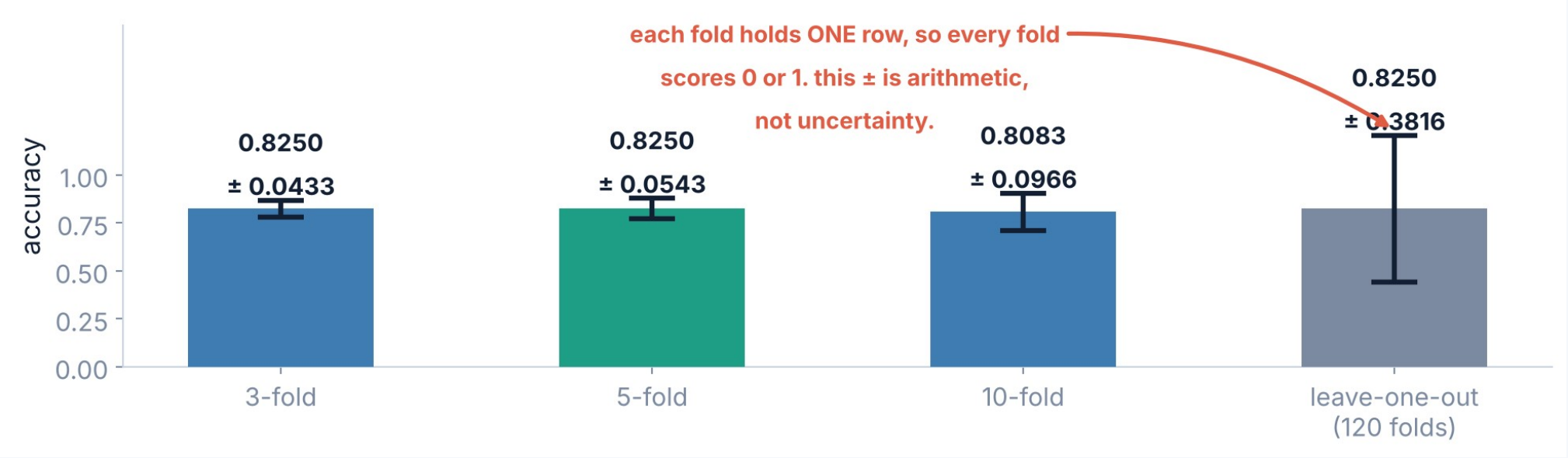
Is it actually more reliable? Measure it.



Twenty single splits ranged over 16.7 points. Ten cross-validations of the same 120 rows ranged over 4.2 — four times tighter, from exactly the same data.

3, 5, 10 OR 120

How many folds should you use?



5 is the default for good reason. 10 holds back only 12 rows per fold here, so each fold's score is noisier — look at how the error bar grows.

A WARNING

Leave-one-out, and why its ± is a lie

What it does

120 folds. Each one trains on 119 students and is tested on the single remaining one.

So each fold's "accuracy" is either 1.0 (got that student right) or 0.0 (got them wrong). There is no third possibility.

It got 99 of the 120 right, giving a mean of 0.8250.

WHY THE ± 0.3816 IS MEANINGLESS

The standard deviation of a list of 0s and 1s is fixed by the mean. With 99 ones and 21 zeros it has to be 0.38 — it would be 0.38 for any model that got 99 right, however stable or unstable that model was.

It is not measuring uncertainty. It is measuring the fact that 0 and 1 are far apart.

When leave-one-out is worth it: when you have so few rows (under about 50) that holding back 20% leaves nothing to test on. Even then, quote the mean and say nothing about the spread. The honest default for anything larger is 5-fold or 10-fold.

Every number that comes out of a tool means something specific. Quoting one without knowing what it measures is how wrong claims get made.

SHUFFLE AND STRATIFY

What happens when the file arrives sorted

Exports are sorted. By date, by customer, by region — or, as here, by the outcome itself. The rows have not changed; only their order has.



Fold 1 contained no passes at all, so the model scored 0.1250 on it. Nothing was wrong with the model. The folds were built wrong.

A REFERENCE

Which splitter, and when

use this	when	why
StratifiedKFold (shuffle=True)	classification — almost always	keeps every fold's class mix matching the whole dataset
KFold (shuffle=True)	regression, where there are no classes to balance	shuffle is still essential; a sorted file is just as dangerous
GroupKFold	several rows belong to the same person, patient or shop	keeps all of one group on one side, so the model cannot study a person and then be tested on them
TimeSeriesSplit	the rows are in time order and you predict the future	always trains on the past and tests on the future — never shuffle time

The last two exist because 'rows it never saw' has to mean genuinely never saw. Two visits from the same patient are not two independent rows.

SECTION 4 OF 5

The leak you cannot see

Cross-validation done correctly, on data that was already spoiled

- 81% on coin flips
- which steps leak
- the rule that explains it

AND THE TRAP IN IT

A reasonable-sounding plan

1 Clean the data

fill the gaps, scale the columns, pick the columns worth keeping

2 Cross-validate

5-fold, stratified, shuffled — everything you just learned

3 Report the mean

with its standard deviation, exactly as told

Every step is correct. The order is wrong, and the order is everything.

Step 1 looked at all 200 rows. By the time step 2 hides 40 of them, the preparation has already carried information out of those 40 and into the model.

This is called leakage. It does not throw an error, it does not look wrong in the code, and it makes your score better — which is why it survives review.

A CONTROLLED EXPERIMENT

Proof, with data that cannot be predicted

200 patients · 5,000 columns of pure random numbers · the label is a coin flip

Nothing in this data predicts anything. An honest score must be about 0.50.

SELECT FIRST, THEN CROSS-VALIDATE	SELECTION INSIDE THE PIPELINE
the best 20 of 5,000 are picked once, using every row	the best 20 are picked again in every fold, from its rows only
0.8150	0.5400

A difference of 27.5 points, entirely invented — and the first number is the one that gets published.

There is no signal in this data. There cannot be — the labels are coin flips. 0.8150 is the sound of the selection step reading the answers.

STEP BY STEP

How the 0.8150 is manufactured

1 5,000 noise columns

by pure chance some of them line up with the coin flips

2 Pick the best 20

chosen using all 200 rows, labels included

3 Hide 40 rows

but those 40 helped choose the columns

4 Score 0.8150

on columns picked to fit the very rows being scored

Why the pipeline version gets 0.5400 instead

Inside each fold the 20 columns are chosen again, using only that fold's 160 training rows. The columns that happened to fit the 40 hidden rows are no longer in the running — so the fiction disappears, and the score falls back to roughly the coin flip it should be.

0.5400 is not quite 0.5000 either. With five folds of 40 rows, that is ordinary noise — and the honest answer is 'this data predicts nothing'.

FOUR STEPS, FOUR ANSWERS

Does every step leak? No. Measured.

Four steps done before the split, each measured

the step before splitting	does it look at y ?	what the leak was worth
scale the columns	no	+0.0000
fill the gaps with the median	no	−0.0067
pick the best 20 of 5,000 columns	YES	+0.3050
pick the best 8 of 60, strong signal	YES	+0.0000

A step that never looks at the answers leaks nothing. A step that chooses using the answers can leak everything.

Scaling leaked exactly nothing. This matters: if you tell students everything leaks, they stop believing any of it. Only one of these four was a disaster.

LOOK AT Y

The rule that predicts which ones are dangerous

A step that never looks at y leaks almost nothing. A step that makes a choice using y can leak everything.

Never looks at y

StandardScaler · SimpleImputer · OneHotEncoder · PolynomialFeatures · VarianceThreshold

Adding 20% more rows barely moves a mean, a median or a standard deviation, so the leaked information is negligible.

Chooses using y

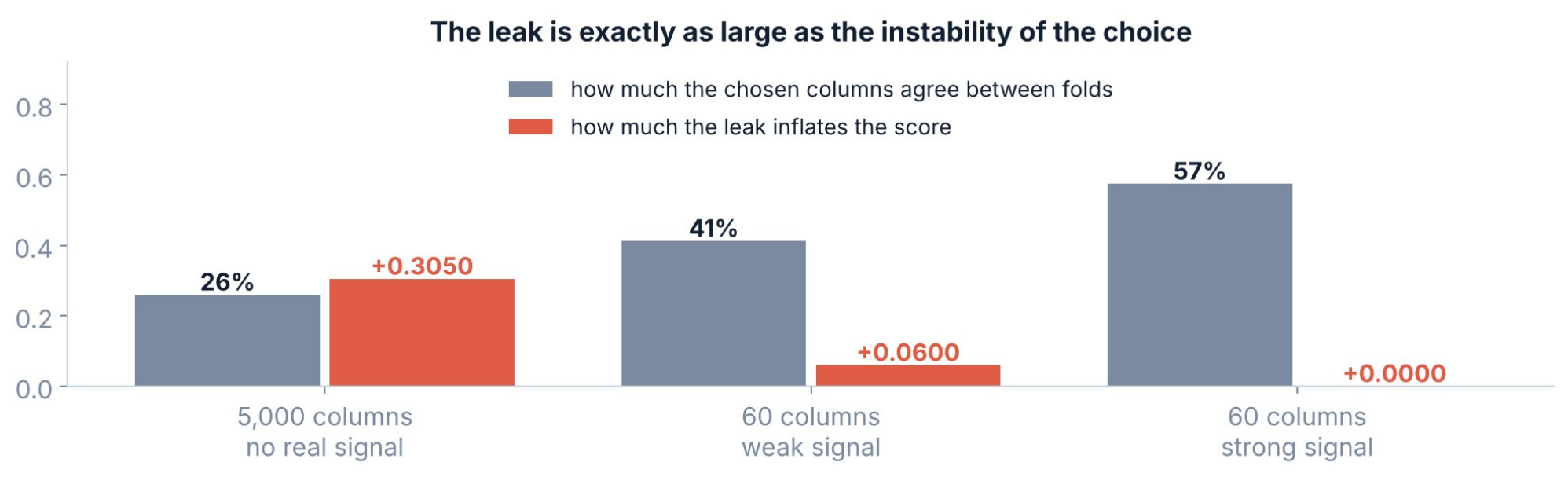
SelectKBest · RFE · "drop the columns that don't correlate with the target" · SMOTE and other resampling · tuning any setting

The choice is itself part of the model, and it was made using the answers.

Note the last item on the right. Choosing max_depth by trying values and keeping the best is a choice made using y — which is why tuning gets its own day, and its own extra split.

THE HONEST NUANCE

And it is only as big as the choice is unstable



When the signal was strong the same 8 columns were picked in every fold, so there was nothing left to leak — and the leak was exactly zero. The danger lives where the evidence is weakest.

NO — AND THAT IS THE POINT

So do you have to judge every step?

You could reason it out every time: does this step look at y, is the choice stable, how many rows do I have.

Or you could put every step inside one object that refits all of them in every fold, and stop thinking about it.

judge each step

slow, and you will be wrong once

→

Pipeline

right by construction

This is the real argument for pipelines, and it is not about scores. It is that correctness stops depending on you remembering.

SECTION 5 OF 5

One object, no leaks

Pipelines, ColumnTransformer, and a real model built end to end

- mixed columns handled
- 900 loan applications
- one file to deploy

THE IDEA

A Pipeline is a list of steps that behaves like a model

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([("scale", StandardScaler()),
                 ("model", LogisticRegression(max_iter=2000))])

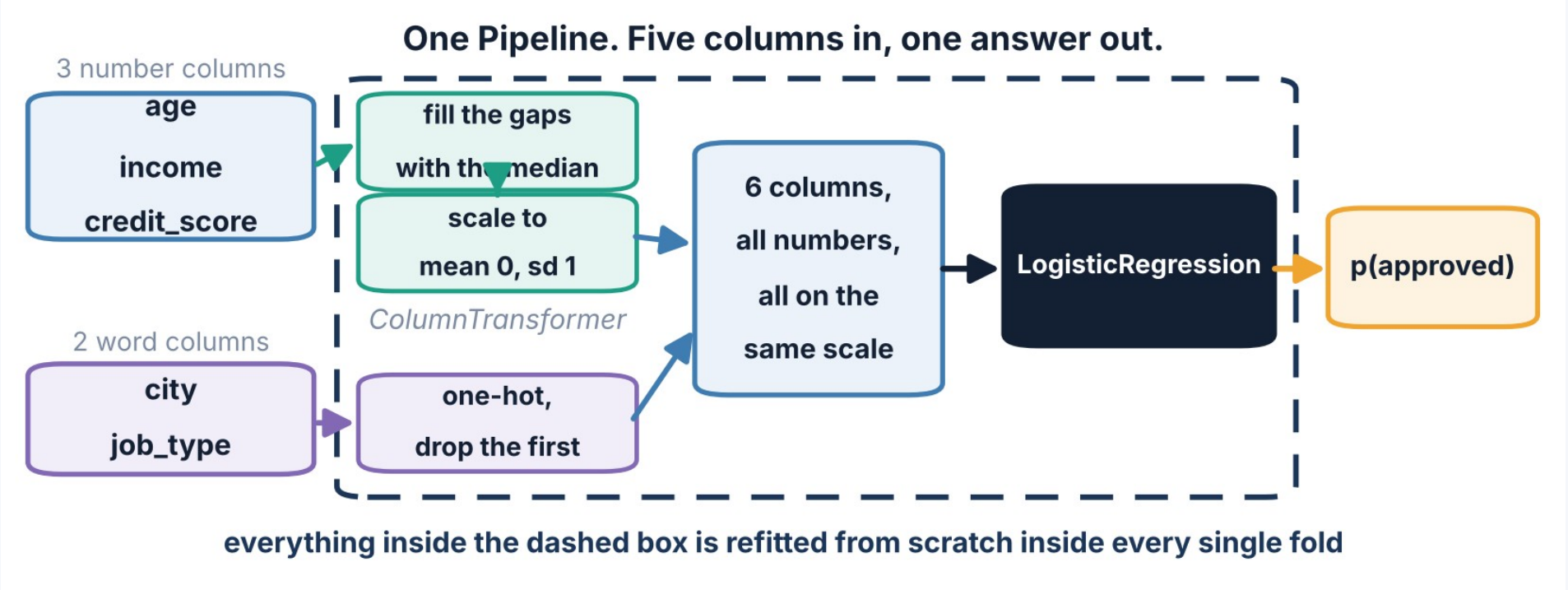
pipe.fit(X_train, y_train)        # fits BOTH steps
pipe.predict(X_new)               # scales X_new, then predicts
```

1. **It has .fit and .predict** — so anywhere a model goes, a pipeline goes — including straight into cross_val_score
2. **Transformers are fitted on train only** — then applied to whatever comes next, with the numbers learned from train
3. **Order is guaranteed** — you cannot scale after predicting, or forget the scaler on new data

"Fitted on train only" is the whole trick. A Pipeline inside cross_val_score refits every step, in every fold, without being asked.

COLUMNTRANSFORMER

Real data has number columns and word columns



The numbers need filling and scaling. The words need one-hot encoding. ColumnTransformer sends each group down its own route and glues the results back together.

THE CODE YOU WILL REUSE

The whole thing, written out

```
num = ["age", "income", "credit_score"]
cat = ["city", "job_type"]

prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc",  StandardScaler())]), num),
    ("cat", OneHotEncoder(drop="first"), cat)])

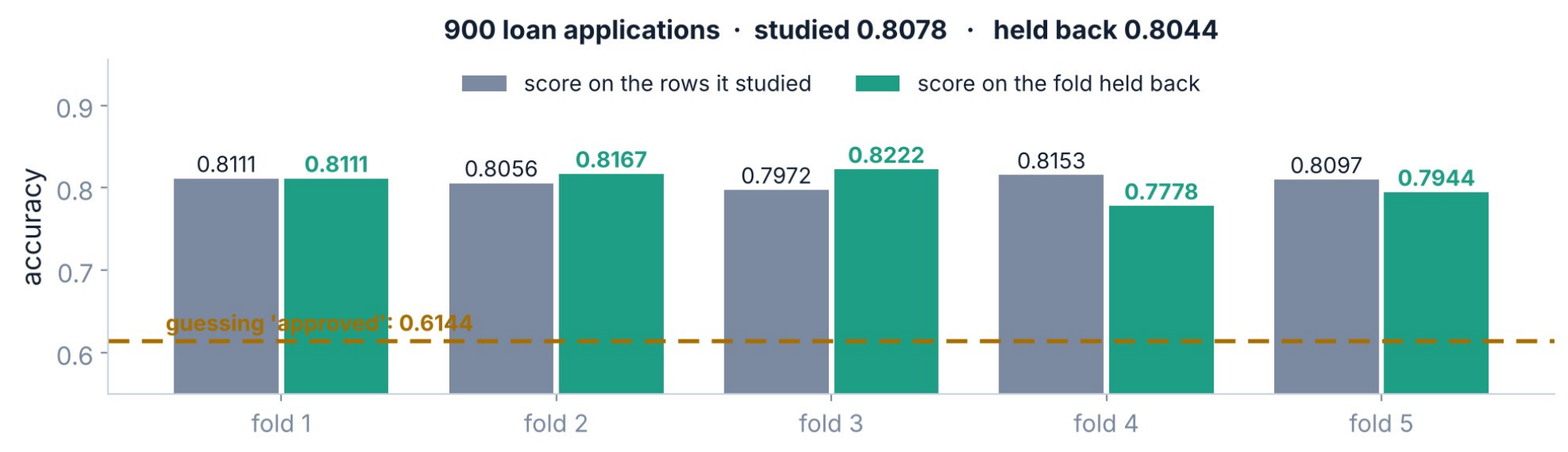
full = Pipeline([("prep", prep),
                 ("model", LogisticRegression(max_iter=2000))])

cv     = StratifiedKFold(5, shuffle=True, random_state=0)
scores = cross_val_score(full, X, y, cv=cv)
```

That last line fits the imputer, the scaler, the encoder and the model five separate times, each on four-fifths of the data. You wrote none of that loop.

THE RESULT

900 loan applications, end to end



Studied 0.8078 · held back 0.8044 · the gap is 0.0034. This model is not overfitting — and that sentence only means something because of everything in the first four sections.

0.8044 against a baseline of 0.6144 — a gain of 19 points, with 76 missing incomes filled and two word columns encoded, all inside the folds.

SO WHY BOTHER?

On this data, by hand gave the same answer

On this data, doing it by hand gave the same answer: 0.8044 either way.

So the Pipeline is not a trick for a better score. Here is what it is actually for.

YOU CANNOT FORGET A STEP	CROSS-VALIDATION IS SAFE	ONE OBJECT TO SAVE
new rows get the same median, the same scaling, the same column order — automatically	every step is refitted inside each fold, without you deciding which ones needed it	joblib.dump(pipe, 'm.joblib') saves the whole thing, not six separate pieces to reassemble

I could have hidden this slide and told you pipelines protect your score. They sometimes do. Mostly they protect you from yourself, and that is worth more.

JOBLIB

Fit it once, save it, use it tomorrow

```
import joblib

full.fit(X, y)                          # now fit on every row you have
joblib.dump(full, "loan_model.joblib")

#  next week, in a completely different program:
model = joblib.load("loan_model.joblib")
model.predict_proba(new_applications)[:, 1]
```

Why fit on everything at the end

Cross-validation was the measurement — it told you to expect about 0.80. The five models it built have done their job and are gone. The model you ship is fitted on all 900 rows.

WHAT GOES IN THE FILE

The median income used for filling (57.9), every column's mean and standard deviation, the category list, the column order, and the coefficients. Save the model alone and every one of those is lost.

A model on its own is not deployable. A pipeline is.

ONE MORE SPLIT

So do you still need a test set?

If you only FIT

Cross-validation is enough on its own. You made no choices using the scores, so the mean is an honest estimate.

If you also CHOOSE

Trying five models, or ten values of max_depth, and keeping the best CV score — that is a choice made using y. Section 4 applies to it.

all your rows

everything you have

→

80% working set

cross-validate inside here as much as you like

→

20% final test

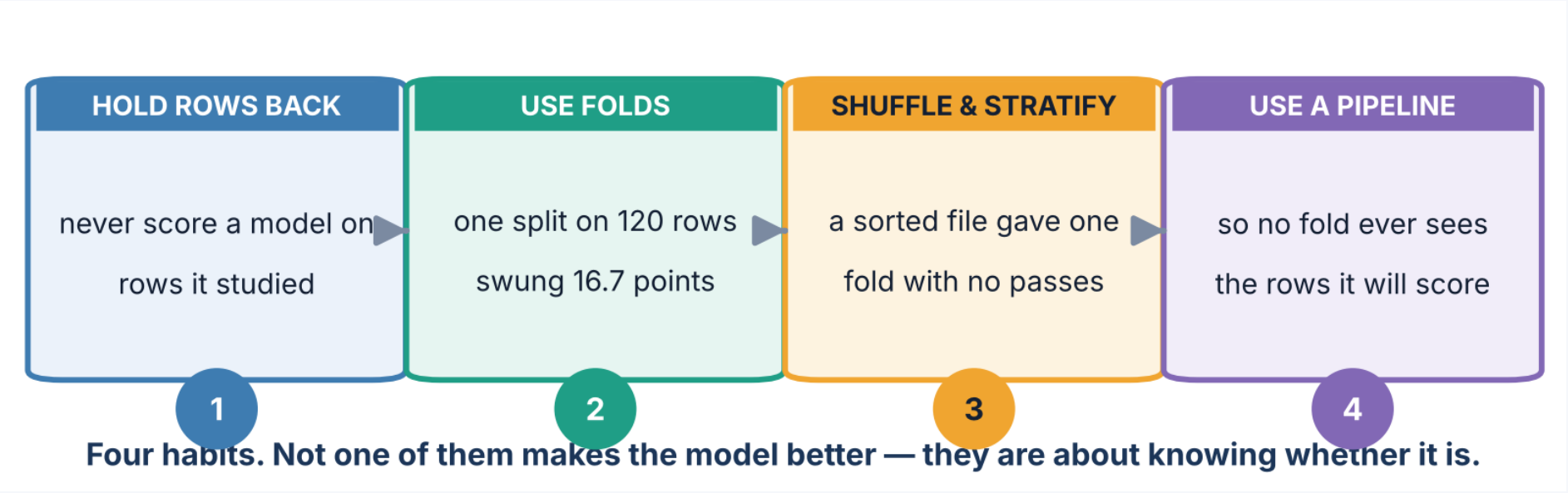
opened once, at the very end

Best of twenty is not an average of twenty. If you pick the highest of twenty cross-validated scores, that winner is high partly on merit and partly on luck — and only rows nothing has touched can tell you how much of each.

You are fitting only, for now, so cross-validation alone is honest. Keep this slide for the tuning day, when you start choosing.

ALL OF TODAY ON ONE PAGE

The four habits



You will use all four for the rest of this course and the rest of your career. Nothing after today changes them — the models get cleverer, the measuring does not.

A CHECKLIST

Mistakes this lecture should stop you making

if you catch yourself…	what is wrong	do this instead
reporting .score(X_train, y_train)	that is the studied rows	score the held-back rows, or cross-validate
trying seeds until one looks good	you are picking the lottery ticket	cross-validate and report mean ± sd
scaling before train_test_split	harmless here, but a bad habit that will bite with other steps	put the scaler in a Pipeline
picking columns before cross-validating	the leak that was worth 27.5 points	put the selector in the Pipeline
KFold on a file you did not shuffle	a fold with no positives in it	StratifiedKFold(shuffle=True)
quoting a mean with no spread	half the result is missing	always quote both

Every one of these makes your number go up. That is what makes them hard to spot — nobody ever double-checks a disappointing result.

WHAT TO DO

Tonight's practice

A — Work it out

8 questions

Fold arithmetic, reading a cross-validation result, spotting the leak on paper

B — Find the mistake

4 questions

Four programs that all report a number that is too good

C — Write the code

5 tasks

Cross-validate properly, then build a full ColumnTransformer pipeline

D — Challenge

2 tasks

Write k-fold yourself from scratch, then audit a model a company is already using

Part D asks you to write the k-fold loop yourself — slicing the row numbers into five groups by hand — and confirm it gives the same five scores scikit-learn gives.

WHAT TO REMEMBER

Five numbers from today

1.0000 — a decision tree on the rows it studied — and 0.6389 on rows it had not, below the 0.7500 you get by guessing

16.7 points — the spread across twenty train/test splits of the same 120 students, with nothing changed but the seed

0.1250 — what one fold scored when the file arrived sorted and nobody passed shuffle

0.8150 — accuracy on labels that were literally coin flips, because the columns were picked before the folds

0.0000 — the leak from scaling before splitting — not every warning you read online is worth the same

Next: evaluation metrics for classification — what to measure once you know how to measure it honestly.